AhamX Bodhi

Where I Becomes Infinite

Foundation Model Comparison

GPT, Llama, and TinyLlama — Module 1.2 LLM Fundamentals

Session Overview

What You Will Learn

- Compare GPT, Llama, and TinyLlama architectures
- Understand key differences in scale and design
- Explore access models: API vs. open weights
- Identify the right model for real-world tasks

Why This Matters

- Three dominant model families in production
- Each serves distinct deployment scenarios
- Guides practical tool selection decisions
- Foundation for all later fine-tuning modules

Learning Objectives

By the End of This Session You Will Be Able To

1. **Identify** the architectural lineage of GPT, Llama, and TinyLlama
2. **Compare** scale, parameter counts, and training data across families
3. **Distinguish** proprietary API models from open-weight alternatives
4. **Select** an appropriate foundation model for a given use-case scenario
5. **Explain** how model size relates to capability and resource requirements

Prerequisites and Context

Prerequisites

- What is a Language Model? (FN-TH-000001)
- Probability and Token Prediction (FN-TH-000002)
- Autoregressive Generation (FN-TH-000003)
- Foundation Models Overview (FN-TH-000005)

What This Unlocks

- Informed API selection in Module 3
- Baseline for fine-tuning decisions in Module 7
- Practical context for scaling laws (Module 9)
- Grounded understanding for agent tool use (Module 5)

The Foundation Model Landscape

GPT Family

OpenAI — Proprietary API

Llama Family

Meta AI — Open Weights

TinyLlama

Open Source — Edge/Local

Three Dominant Families

Foundation models differ in **access model**, **parameter scale**, and **deployment context** — choosing correctly is a core practitioner skill.

GPT Family --- Architecture and Scale

GPT Architecture

- Decoder-only Transformer
- Autoregressive next-token prediction
- RLHF alignment (InstructGPT, GPT-4)
- Massive scale: GPT-3 at 175B parameters
- Context windows up to 128K tokens (GPT-4o)

Key Characteristics

- Accessed exclusively via OpenAI API
- Closed weights — no local deployment
- Best-in-class instruction following
- Multimodal capability (GPT-4V, GPT-4o)
- Per-token pricing model

GPT Family --- Versions and Milestones

Model	Parameters	Year	Key Innovation
GPT-1	117M	2018	Unsupervised pre-training
GPT-2	1.5B	2019	Zero-shot task transfer
GPT-3	175B	2020	In-context few-shot learning
InstructGPT	1.3B–175B	2022	RLHF alignment introduced
GPT-4	Undisclosed	2023	Multimodal, reasoning leap
GPT-4o	Undisclosed	2024	Omni: text, audio, vision

Practitioner Note

GPT models set the benchmark. Understanding their evolution clarifies what “alignment” and “instruction tuning” mean in practice.

Llama Family --- Meta's Open-Weight Strategy

Architecture Improvements

- Decoder-only Transformer (GPT-style base)
- Grouped Query Attention (GQA) in Llama 2+
- RoPE (Rotary Position Embedding)
- RMSNorm instead of LayerNorm
- SwiGLU activation function

Why Llama Changed the Field

- Fully open weights — deploy anywhere
- Community fine-tuning ecosystem (Alpaca, Vicuna)
- Scales from 7B to 405B parameters
- Commercial licenses available (Llama 2, Llama 3)
- Foundation for thousands of derived models

Llama Family --- Versions Comparison

Model	Sizes	Context	Key Feature
Llama 1	7B–65B	2K tokens	First open research release
Llama 2	7B–70B	4K tokens	Commercial license, RLHF chat
Llama 3	8B–70B	8K tokens	Improved tokenizer (128K vocab)
Llama 3.1	8B–405B	128K tokens	Tool use, multilingual support

Open-Weight Advantage

Llama models can be downloaded, fine-tuned on private data, and deployed on-premises — critical for regulated industries and privacy-sensitive applications.

TinyLlama --- Efficient Models for the Edge

TinyLlama Specification

- 1.1B parameters only
- Trained on 3 trillion tokens
- Same architecture as Llama 2
- Context length: 2048 tokens
- Apache 2.0 license — fully open

Why TinyLlama Matters

- Runs on CPU, Raspberry Pi, mobile devices
- Extremely fast inference on edge hardware
- Research into efficient pretraining
- Entry point for custom LLM training
- Demonstrates data-efficient scaling

Feature Comparison --- GPT vs. Llama vs. TinyLlama

Feature	GPT (OpenAI)	Llama (Meta)	TinyLlama
Access Model	Proprietary API	Open weights	Open weights
Smallest size	1.3B (GPT-3.5)	7B (Llama 1/2/3)	1.1B
Largest size	Undisclosed	405B (Llama 3.1)	1.1B
Local deployment	No	Yes	Yes
Commercial use	Via API terms	Llama 2/3 license	Apache 2.0
Instruction tuning	Yes (InstructGPT)	Yes (Llama 2 Chat)	Via fine-tuning
Multimodal	Yes (GPT-4V, 4o)	Llama 3.2 Vision	No
Cost model	Per-token billing	Compute cost only	Minimal

Architecture at a Glance

GPT Family	Llama Family	TinyLlama
Decoder-only	Decoder-only + GQA	Decoder-only (Llama 2)
Proprietary / Closed	Open weights	Open weights
175B – undisclosed	7B – 405B	1.1B
API only	Local / cloud	Edge / on-device

Application 1 --- Customer Service Chatbots

GPT-4o via API

- Companies: Klarna, Intercom, Zendesk
- Plug-in via OpenAI API
- Best-in-class language understanding
- Rapid deployment, no ML expertise needed
- Cost: $0.005 per 1K output tokens

Llama 2 On-Premise

- Healthcare/finance: data stays on-site
- Fine-tune on company knowledge base
- One-time infrastructure cost
- Full control over model behaviour
- Example: hospital triage assistant

Application 2 --- Code Generation and Developer Tools

GPT-4 in Production

- GitHub Copilot uses OpenAI models
- Cursor IDE integrates GPT-4o
- Multi-file context understanding
- Inline chat, explanations, refactoring

CodeLlama (Llama-based)

- Meta's code-specialised Llama variant
- Infilling: insert code in the middle
- Self-hosted in enterprise environments
- Supports 80+ programming languages

Application 3 --- Edge AI and On-Device Intelligence

TinyLlama in the Wild

- **IoT devices:** Smart home automation pipelines with local NLP
- **Mobile apps:** On-device query answering without network round trips
- **Robotics:** Low-latency command parsing on embedded hardware
- **Education tools:** Offline tutoring assistants on low-cost laptops

Why Edge Matters

Privacy, latency, and connectivity constraints make TinyLlama the only viable option in many real-world scenarios.

Application 4 --- Research and Experimentation

Open Models Enable Research

- **Interpretability:** Researchers inspect Llama weight activations directly
- **Fine-tuning studies:** Ablation experiments on small datasets
- **Benchmarking:** Reproducible evaluations on fixed weights (MMLU, HumanEval)
- **Custom pre-training:** TinyLlama as efficient baseline for new domain corpora
- **Alignment research:** Test RLHF, DPO, ORPO without API restrictions

Contrast with GPT

GPT models cannot be inspected, fine-tuned locally, or reproduced — open-weight models are essential for rigorous scientific research.

Application 5 --- Retrieval-Augmented Generation

GPT + RAG (LangChain)

- LlamaIndex + GPT-4 for document QA
- Long context handles large retrievals
- Enterprise search (Microsoft Copilot)
- Reduces hallucination on factual queries

Llama + RAG (Self-Hosted)

- Ollama + Llama 3 for private document search
- No data leaves the organisation
- LangChain / LlamaIndex support Llama fully
- Ideal for legal, medical, defence sectors

How to Choose the Right Foundation Model

Model Selection Decision Guide

- **Best performance, no local deployment needed?** → **GPT-4o** (OpenAI API)
- **Data must stay on-premise, large scale needed?** → **Llama 3** (self-hosted)
- **Resource-constrained or edge device?** → **TinyLlama** (local CPU/GPU)
- **Research or open experimentation?** → **Llama family** (open weights)
- **Rapid prototyping, no infra budget?** → **GPT-4o-mini** (cheapest API)

Key Decision Factors

Evaluate: **data privacy**, **deployment environment**, **budget**, and **task complexity** — in that order.

Parameter Count and Capability

What Parameters Determine

- **Capacity:** More parameters → more knowledge stored
- **Reasoning:** Larger models generalise better to novel prompts
- **Compute cost:** Inference FLOPS scale roughly linearly with size
- **Memory:** A 7B model in FP16 requires ≈14 GB VRAM
- **Latency:** Models ≤1B respond in milliseconds on CPU

The Size–Cost–Quality Triangle

No model wins on all three simultaneously. Practitioners navigate this triangle based on application constraints.

Skills Developed in This Concept

Technical Skills

- Model family identification
- Architecture vocabulary (GQA, RoPE, RMSNorm)
- Access pattern differentiation (API vs. weights)
- Resource estimation for deployment

Applied Skills

- Use-case to model matching
- Trade-off analysis (cost, privacy, quality)
- Reading model documentation and specs
- Communicating model choices to stakeholders

Professional Role Relevance

Role	How This Concept Applies
AI Engineer	Selects and integrates foundation models in production pipelines
ML Engineer	Evaluates models for fine-tuning feasibility and resource planning
Data Scientist	Uses GPT/Llama APIs for rapid prototyping and experimentation
Software Developer	Integrates OpenAI API or Ollama into applications
Product Manager	Understands model trade-offs to guide build-vs-buy decisions
Research Scientist	Studies open-weight models for interpretability and alignment work

Knowledge Graph Positioning

Requires (Prerequisites)

- What is a Language Model? (FN-TH-000001)
- Probability and Token Prediction (FN-TH-000002)
- Autoregressive Generation (FN-TH-000003)
- Foundation Models Overview (FN-TH-000005)

Enables (Unlocks)

- API Integration and Parameter Tuning (M3)
- Practical Inference Optimisation (M3)
- Fine-Tuning Fundamentals (M7)
- LLMOps and Deployment (M9)

Common Misconceptions

Misconceptions to Avoid

- **“Bigger is always better”** — TinyLlama outperforms GPT-3 on some narrow tasks
- **“Open = worse quality”** — Llama 3 70B matches GPT-3.5 on many benchmarks
- **“GPT and Llama are architecturally different”** — Both are decoder-only Transformers
- **“Free models have no cost”** — Open-weight models require compute infrastructure
- **“One model fits all use-cases”** — Model selection is always context-dependent

Summary

Key Takeaways

1. **GPT, Llama, TinyLlama** represent three distinct access and scale philosophies
2. **All three** are decoder-only Transformers — same core architecture
3. **Access model** (proprietary API vs. open weights) is the primary differentiator
4. **Model size** governs capability, memory, cost, and latency simultaneously
5. **Task requirements** — not marketing — should drive model selection

Next Steps

Immediate Next Concepts

- Module 1 Assessment (AI-GN-FN-AS-000001)
- Mini Project 1: Environment Setup
- Module 2: Transformer Architecture

Recommended Practice

- Try GPT-4o mini via OpenAI Playground
- Run TinyLlama locally using Ollama
- Compare outputs on identical prompts

Thank You

Foundation Model Comparison — GPT, Llama, and TinyLlama